

Exercise 3: Sensitivity analysis and Random Fields in Uncertainty Quantification

Group 6

1 Sensitivity Analysis

In this report, we analyze the sensitivity of a linear damped oscillator. We focus on computing and comparing the first and total order Sobol' indices using two different approaches: a Monte Carlo-based method introduced in Saltelli et al., and a pseudo-spectral approach based on gPCE.

We used the Monte-Carlo method introduced in the paper: let A and B be two $N \times k$ matrices, where the N rows represent the number of samples and k columns represent the stochastic dimensionality of our model. Furthermore, let f denote our model. The method to compute the i -th Sobol' index of first order, $i = 1, \dots, k$ is:

$$D_i = \frac{1}{N} \sum_{j=1}^N f(B)_j \left(f(A_B^{(i)})_j - f(A)_j \right).$$

Here, the notation $A_B^{(i)}$ represents the matrix A with its i -th column replaced with the i -th column from B , and $(B)_j$ denotes the j -th row of matrix B .

In our case, f is the model of the damped linear oscillator and the number of uncertain input parameters is $k = 5$, corresponding to c , k , f , y_0 , and y_1 . For a fair comparison with the pseudo-spectral approach using a full tensor grid with $K = 4$, we use $N = 4^5 = 1024$ Monte Carlo samples. Thus, the input matrices A and B have dimensions 1024×5 .

We plot the first order and total indices for different approaches and discuss the pros and cons of different approaches:

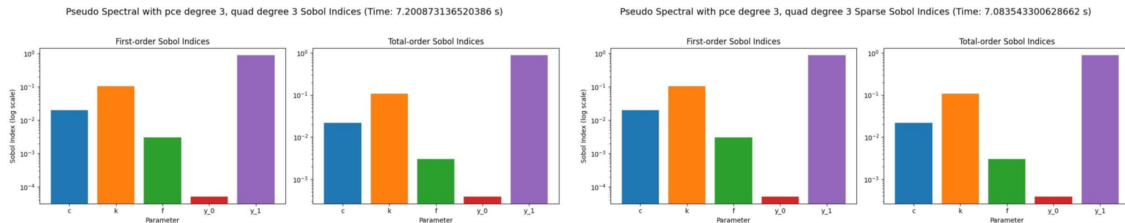


Figure 1: PCE deg=3 with quadrature deg=3 on full grid vs. sparse grid

Overall, the sensitivity of parameters remains consistent across both methods: the parameter y_1

has the highest influence on the output variance, followed by k , c , and f , while y_0 has a negligible effect. There is very little difference in the computed Sobol' indices between the full and sparse grid approaches. Given that sparse grids reduce the number of required model evaluations while preserving accuracy, they are a more efficient choice for uncertainty quantification in our case.

We see a similar situation with higher degree in polynomial expansion and quadrature:

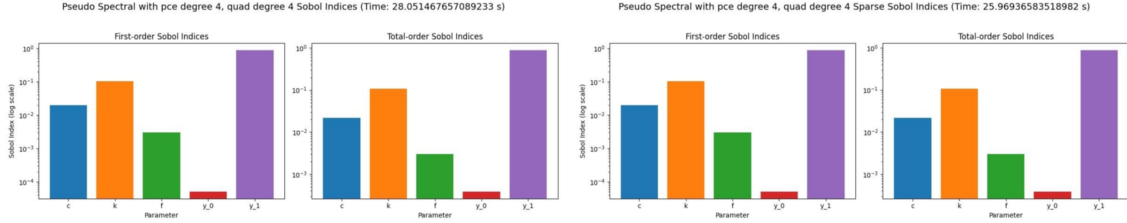


Figure 2: PCE deg=4 with quadrature deg=4 on full grid vs. sparse grid

Increasing the polynomial and quadrature degrees from 3 to 4 does not significantly change the Sobol' indices, as the parameter sensitivity remains nearly identical. Once again, sparse grids achieve comparable accuracy to full grids at reduced computational cost.

Computing Sobol' indices using Monte-Carlo sampling produces slightly different results:

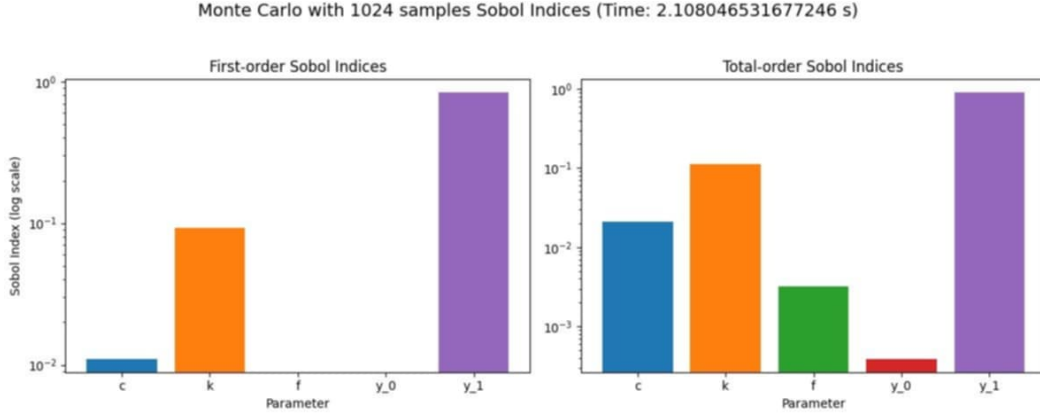


Figure 3: Monte-Carlo approach to Sobol' indices

Compared to the pseudo-spectral methods, the Monte Carlo-based Sobol' indices show more fluctuation, especially for the less sensitive parameters c , f , and y_0 . This is expected due to the stochastic nature and slower convergence of Monte Carlo methods, especially with a moderate sample size ($N = 1024$). While the dominant parameters (k and y_1) are correctly identified, the results are generally noisier and less precise than those from the deterministic pseudo-spectral approaches.

Computational cost of Monte-Carlo approach

For the Monte-Carlo method, the model is evaluated on three sets of samples: the base matrices A and B , each of size $N \times k$, and k additional matrices $A_B^{(i)}$, each also of size $N \times k$. This results in a total of N model evaluations for A , N for B , and $N \cdot k$ for all $A_B^{(i)}$ combined. Therefore, the total computational cost in terms of ODE solves is:

$$\text{Cost}_{\text{MC}} = N(k + 2)$$

In our case, with $N = 1024$ and $k = 5$, this results in $1024 \cdot (5 + 2) = 7168$ ODE solves.

The Monte Carlo methods to estimate Sobol indices ran in about 2 seconds, while the quadrature/PCE method took about 28 seconds. MC is faster because its model evaluations are parallel, since each time the oscillator is evaluated with the given input parameters is independent, so the computations can be distributed among CPU cores. With the pseudo-spectral approach, first we need to compute a global quadrature and then solve for coefficients, which is a lot of linear algebra compute before evaluating the model.

2 Covariance functions

In this section, we work with two Gaussian random fields defined on $[0, 1]^2$, each given in terms of its mean $m(x)$ and covariance function $C(x, y)$. We fix the mean to be constant, $m(x) = 0.1$, and consider two covariance functions,

$$C_1(x, y) = \exp\left(-\frac{\|x - y\|_2}{l}\right), \quad l \in \mathbb{R} \quad (1)$$

$$C_2(x, y) = \exp\left(-\frac{\|x - y\|_2^2}{2l^2}\right), \quad l \in \mathbb{R} \quad (2)$$

Let $G^{(1)}$ and $G^{(2)}$ be the Gaussian random fields with the respective covariance functions.

We discretized the unit square $[0, 1]^2$ using a 50×50 grid, placing each point in the center of its corresponding cell, resulting in 2500 grid points. We evaluated each of the covariance functions C_1, C_2 on all grid points using pairwise distances between the points, resulting in two covariance matrices,

$$\hat{C}_1, \hat{C}_2 \in \mathbb{R}^{2500 \times 2500}.$$

In order to perform Cholesky decomposition on $\hat{C}_1, \hat{C}_2$, the matrices must be positive definite. Hence we added a small regularization term to ensure numerical stability:

$$\hat{C}_j + \varepsilon I \rightarrow \hat{C}_j, \quad \varepsilon = 10^{-7}$$

for $j \in \{1, 2\}$.

Then we performed Cholesky decomposition on $\hat{C}_1, \hat{C}_2$,

$$\hat{C}_1 = L_1 L_1^T, \quad \hat{C}_2 = L_2 L_2^T,$$

to obtain L_1, L_2 , the lower triangular Cholesky factors. We sampled three times from each Gaussian random field, where each sample is given by

$$G_j^{(i)} = \hat{m} + L_j \Psi, \quad \Psi \sim \mathcal{N}(0_{2500}, I_{2500}),$$

with $j \in [1, 2]$. Below, we show visualizations of the samples from each random field.

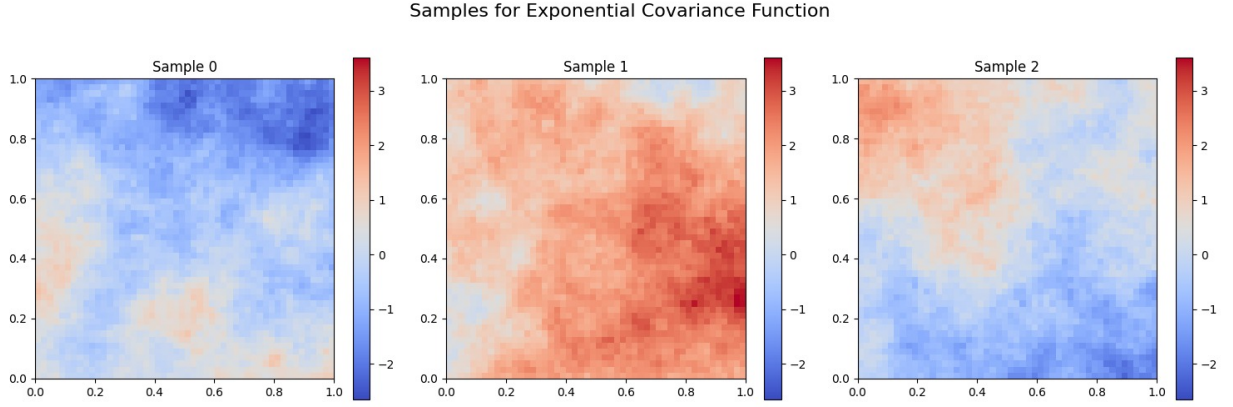


Figure 4: Samples from a Gaussian field with exponential covariance function, $G^{(1)}$.

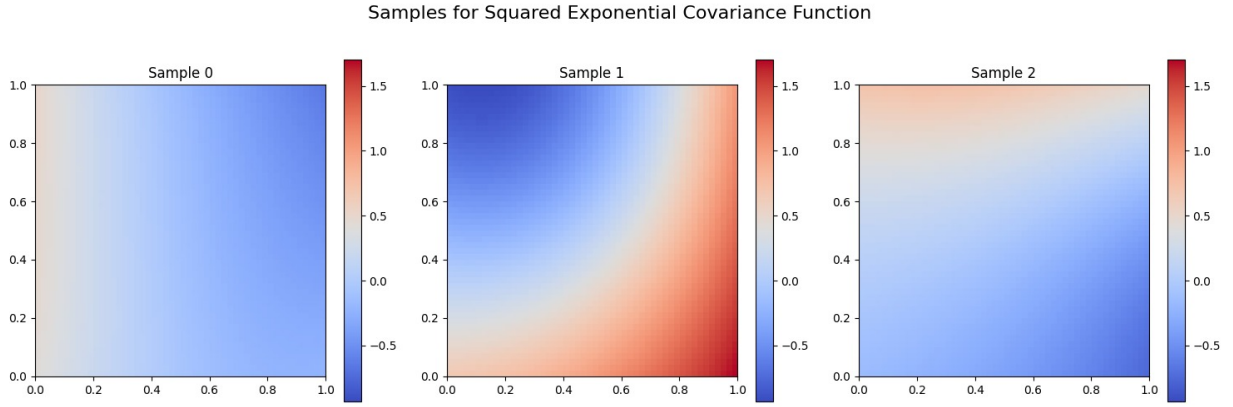


Figure 5: Samples from a Gaussian field with squared exponential covariance function, $G^{(2)}$.

Comparing Figure 4 and Figure 5, we see that all samples from $G^{(1)}$ have rough texture, with abrupt changes in values among nearby discretization cells. It looks like noisy interpolation of an image. We can see that the field $G^{(1)} : \mathbb{R}^2 \rightarrow \mathbb{R}$ has high local variation and is not differentiable.

On the other hand, the samples from $G^{(2)}$ have a very smooth texture with large gradients varying over $[0, 1]^2$. This means that values change continuous among nearby regions in the domain.

In a sense, $G^{(1)}$ preserves more precise information, while $G^{(2)}$ smooths out the values at the loss of detail.

For modeling, the choice of covariance function should be related to the assumed regularity of the quantity of interest. We hypothesize that $G^{(1)}$ could be more useful to model discrete phenomena with non-smooth behavior, such as noise levels in an area or 2-dimensional rough textures, while $G^{(2)}$ could be used to model smooth phenomena, like temperature distributions on 2-dimensional surfaces or electromagnetic fields.

3 The Wiener process

Assignment 3.1

In this exercise, we examined the properties and numerical representation of the Wiener process. Specifically, we explored how the Karhunen-Loève (KL) expansion can be used to approximate realizations of the Wiener process and studied its convergence behavior for varying numbers M of KL terms. Additionally, we present a realization of the Wiener process using its definition.

We used the definition of the Wiener process in the time interval $T = [0, 1]$. We discretized T with $N = 1000$ points, which means each time interval was of length $dt = 0.001$. Then we can compute the realization as

$$dW := W_{t+dt} - W_{dt} \sim \mathcal{N}(0, dt)$$

and in order to obtain a time series for W_t we sum up all the previous realizations. An example of the Wiener process W_t is shown here:

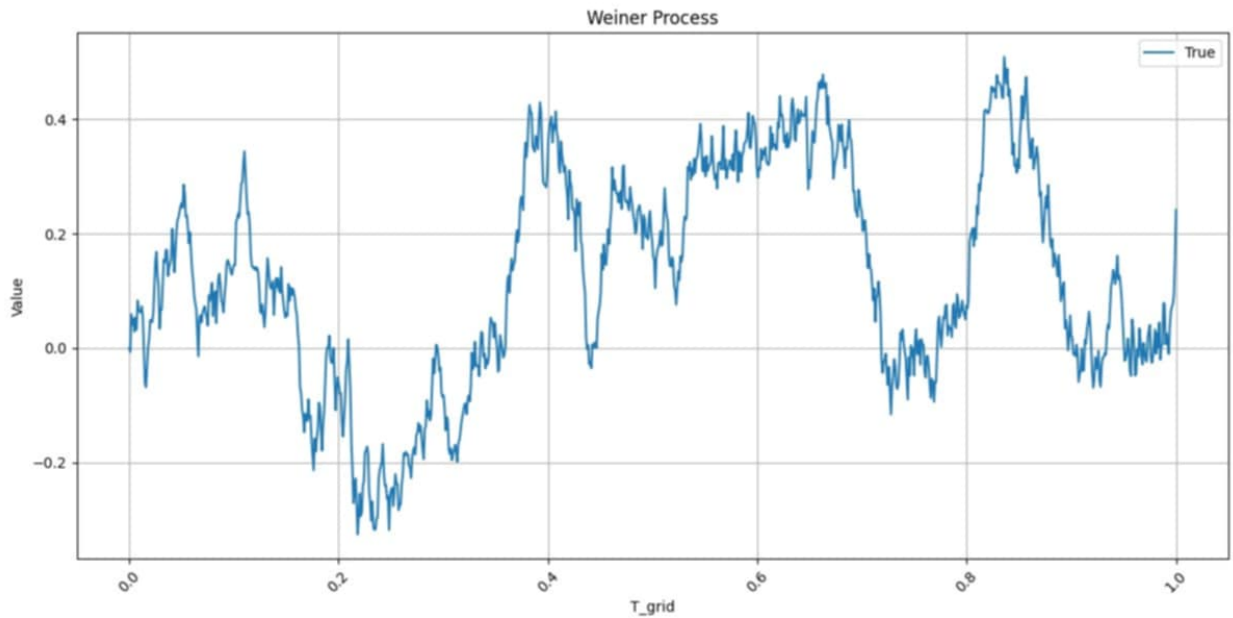


Figure 6: Realization of the Wiener process using its definition

Here, the parameter N denotes the number of time discretization points used to represent the continuous time interval $[0, 1]$. In other words, N determines the temporal resolution of the process: a larger N leads to a finer discretization and a more accurate representation of the continuous-time Wiener process.

In contrast, the Karhunen-Loeve (KL) expansion approximates the Wiener process using the orthogonal eigenfunctions ψ_n and eigenvalues λ_n found via the 2-nd Fredholm equation

$$\int_{\mathcal{T}} C(t, s) \psi_n(t) dt = \lambda_n \psi_n(s),$$

where in our case $C(t, s) = \min(t, s)$. The approximation is optimal for the Mean-Squared Error in the space $L^2([0, 1])$, so

$$\lim_{M \rightarrow \infty} \mathbb{E} \left[\left(W_t - W_t^{(M)} \right)^2 \right] = 0 \quad \forall t \in [0, 1],$$

where M represents the KL approximation to M terms:

$$W_t = \sum_{m=1}^{\infty} \sqrt{\lambda_m} \phi_m(t) \zeta_m \approx \sqrt{2} \sum_{m=1}^M \frac{\sin((m + 0.5)\pi t)}{(m + 0.5)\pi} \zeta_m, \quad \zeta_m \sim \mathcal{N}(0, 1).$$

Thus, M represents the truncation of the KL expansion in the $L^2([0, 1])$ space to the first M eigenfunctions/values. We can see that the higher eigenvalues quickly become very small:

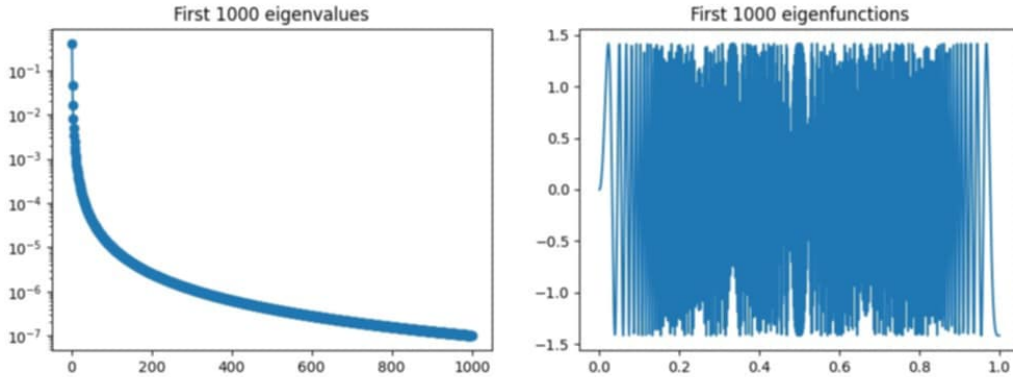


Figure 7: Eigenvalues quickly tend to 0

This means that we do not need many eigenfunctions to accurately approximate the process, as the higher terms in the series provide little additional accuracy.

We plot the series for $M \in \{10, 100, 1000\}$ terms:

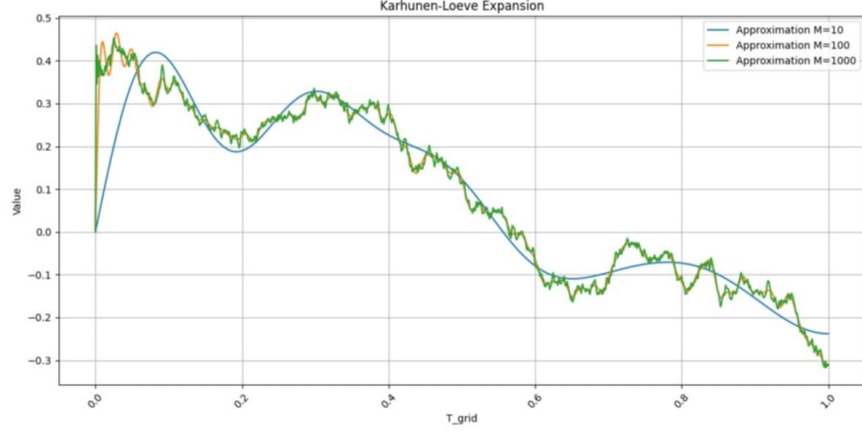


Figure 8: KL expansion for different number of summation terms

We clearly see that the series converges relatively quickly, and the gain in approximation from 100 to 1000 is (almost) negligible. The realization of the Wiener process generated using its definition appears much rougher and more irregular than the smoother approximations obtained via the Karhunen–Loève expansion. This difference arises because the KL expansion converges in the mean-square sense L^2 , not pointwise.

Assignment 3.2

We consider the second-order linear damped oscillator for $t \in [0, 10] = T$:

$$\begin{cases} \frac{d^2 y}{dt^2}(t) + 0.5 \frac{dy}{dt}(t) + 2y(t) = f(t) \cos(\omega t), \\ y(0) = 0.5, \\ \frac{dy}{dt}(0) = 0, \end{cases} \quad (3)$$

where $f(t)$ is stochastic. To analyze uncertainty propagation through the oscillator, we model the source term $f(t)$ as a Wiener process with mean $\mu = 0.5$, so that in general,

$$f(t) = \mu_f + W_t.$$

In particular, we generated the Wiener process using its definition, as well as KL expansion truncated at $M = [5, 10, 100]$.

We discretized T with $N = 1000$ points, so each interval was of length $dt = 0.01$. For the standard Wiener process, we generated 1000 sample paths by sampling

$$dW := W_{t+dt} - W_t \sim \mathcal{N}(0, dt),$$

and to create a time series for each W_t we summed up all previous realizations.

For each $M = [5, 10, 100]$ we generated the KL expansion, as in the previous exercise. All

realizations were stored as callables, which evaluate $f(t)$ at any time grid point.

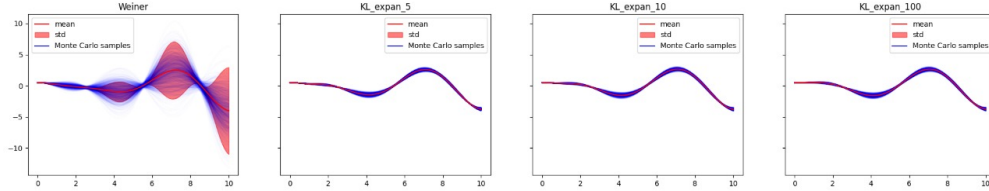
To propagate the uncertainty through the model, we solved the oscillator numerically for each realization of $f(t)$ using odeint functionality. In Table 1, we provide the mean and variance across all trajectories $y(t)$ at $t = 10$ for each method.

Method	Mean of $y(10)$	Variance of $y(10)$
Wiener	-4.00	6.98
KL, $M = 5$	-3.76	0.023
KL, $M = 10$	-3.77	0.023
KL, $M = 100$	-3.75	0.024

Table 1: Mean and variance of $y(10)$ for the Wiener process and truncated KL expansions.

In Table 1, the means given by the KL expansions approach the mean computed using the Wiener definition. The KL expansion produces smoother values of $y(t)$ as shown in the plot below. Hence KL expansions for small M have significantly less variance than the one given by the Wiener definition. As M increases, mean and variance converge to those obtained through the Wiener process.

For the Wiener process, we generated 1000 independent sample solutions $y(t)$, each given by the realization of $f(t)$. By plotting a large number of realizations, we can see that the randomness in $f(t)$ has a large effect on the variability and standard deviation of the solution trajectories at different times.



While Monte Carlo is the standard approach to compute mean and variance of the solution, it is computationally expensive. Instead we could use approaches like generalized polynomial chaos expansion or Galerkin method. We know that $f(t) \approx \mu_f + \sum_{m=1}^M \sqrt{\lambda_m} \phi_m \zeta_m$, $\zeta_m \sim \mathcal{N}(0, 1)$. To use PCE method, we could expand the solution $y_0(t, \zeta)$ as

$$y_0(t, \zeta) \approx \sum_{n=0}^{N-1} \hat{y}_n(t) \varphi_n(\zeta),$$

where φ_n are orthonormal Hermite polynomials up to degree $N - 1$. We would calculate the coefficients $\hat{y}_n(t)$ using the pseudo-spectral approach.

To propagate uncertainty of $f(t)$ through the model using Galerkin approach, we would transform our oscillator into a coupled system of first-order ODEs, determine the exact polynomial chaos

expansion $y(t)$ and $\frac{d}{dt}(y(t))$, that is

$$y(t, \zeta) \approx \sum_{n=0}^{N-1} \hat{y}_n(t) \varphi_n(\zeta), \quad \frac{d}{dt}(y(t, \zeta)) = \sum_{n=0}^{N-1} \frac{d}{dt}(\hat{y}_n(t)) \varphi_n(\zeta).$$

Then we would use initial conditions to compute $y(0, \zeta) = y(0) = 0.5$, $\frac{d}{dt}(y(0, \zeta)) = y(1) = 0$, project our initial condition expansions onto the normalized Hermite polynomials, use orthonormality to find result for $\hat{y}_i(0)$ and $\frac{d}{dt}(\hat{y}_j(0))$ for all j , plug the expansions for $y(t, \zeta)$, $\frac{d}{dt}(y(t, \zeta))$ into our model, project again onto the Hermite polynomials and finally use orthonormality to find $2N$ coupled ODEs for the coefficients $\hat{y}_n(t)$. Finally we would compute expectation and variance as in the pseudo-spectral approach.

While Monte Carlo is very straightforward, methods like gPCE and Galerkin can offer computational savings for such an uncertainty propagation problem.